

Lab 6 Fillable Worksheet

Malware Behavior, Persistence & Evasion

Use this worksheet while completing Lab 6. Fill in each section as you analyze the malware sample. When finished, choose **Print / Save as PDF** to submit your completed handout.

Safety reminder: Analyze malware only inside an isolated virtual machine or a controlled sandbox such as **ANY.RUN**. Never run malware on your host system.

Student Information

Student name

Tyler Vander Mooren

Date

04 / 20 / 2026



Course / section

CYB310-002

Instructor

Troy Adams

Sample Identification

Malware sample name

Agenta Tesla / Stealer

Source repository

MalwareBazaar

File name

Remittance Receipt.JS

SHA-256 hash

e060c3b7f6e59dfb5fdd8fa268dad27a31f1c

Environment Preparation

Preparation item	Done	Notes
Created a clean VM snapshot before execution	☐	Windows 10 VM using VirtualBox.
Configured isolated or controlled networking	☐	VM configured using NAT.

Preparation item	Done	Notes
Disabled shared folders and unsafe host integration	☐	
Started monitoring tools before execution	☐	
Recorded a baseline for startup and persistence locations	☐	X32dbg for debugging, Wireshark for network capture and ProcMon. Autoruns to identify startup and persistence locations.

Behavior After Execution

Behavior category	Observed evidence
New or child processes	WScript.exe launched powershell.exe, and powershell.exe spawned conhost.exe. ANY.RUN showed the chain wscript.exe -> powershell.exe -> conhost.exe, and the local ProcMon log showed the same launcher behavior during execution. WinRAR.exe also appeared as the archive opener before script execution.
Files created, dropped, or modified	The malware caused creation or modification of PowerShell policy test files in %LOCALAPPDATA%\Temp, updated the PowerShell ModuleAnalysisCache, and dropped or created C:\Users\Public\Mands.png and C:\Users\Public\Vile.png. These were the clearest file-system artifacts observed.
Registry keys written or changed	ANY.RUN showed these notable registry writes: HKCU\SOFTWARE\Microsoft\PowerShell\1\ShellIds\Microsoft.PowerShell with ExecutionPolicy = Bypass HKCU\SOFTWARE\Microsoft\Windows Script\Settings\Telemetry\wscript.exe with JScriptSetScriptStateStarted In the VM ProcMon log, PowerShell also accessed HKLM\SOFTWARE\Microsoft\WBEM and HKLM\System\CurrentControlSet\Services\Tcpip\Parameters.
Network connections or beaconing	The strongest malicious network indicators were PowerShell connections to ip-api.com (208.95.112.1) and ftp.ienco-gr.com (68.65.121.198) on ports 21 and 12081. The sandbox also showed many Microsoft and Live service requests, but those were largely whitelisted background or system traffic. The malicious-looking behavior was the external IP / hosting lookup plus FTP-based exfiltration.
Unexpected delays, loops, or restarts	No clear sleep delay was directly observed.

Summarize what the malware appeared to do in the first few minutes after execution

After execution the JavaScript sample ran through WScript.exe and launched an obfuscated PowerShell payload. The PowerShell process used encoded content, modified execution policy behavior, created temporary script artifacts, touched browser related and PowerShell-related locations, checked the external environment through ip-api.com and attempted outbound communication to an FTP host associated with suspected exfiltration. Overall, the first few minutes looked consistent with a credential-stealing loader associated with Agent Tesla.

Persistence Mechanisms

Persistence location or method	Evidence observed
Run or RunOnce registry keys	No confirmed malicious Run or RunOnce persistence entry was identified in the Autoruns before/after comparison.
Startup folders	No confirmed malicious file or shortcut was added to the user or system Startup folders.
Scheduled tasks	No confirmed malicious scheduled task was identified in the before/after Autoruns exports.
Windows services	No confirmed malicious service installation was identified in the before/after Autoruns exports.
WMI or other persistence mechanisms	No confirmed WMI persistence event subscription or other durable persistence artifact was isolated from the provided logs.

Did the malware establish persistence? If yes, explain briefly

No persistence was clearly confirmed from before/after Autoruns comparison.

Evasion and Anti-Analysis

Technique or indicator	Observed evidence
Sleep delays or timing tricks	Not directly observed.
Debugger detection	Not directly observed.
Virtual machine detection	Not directly observed.
Sandbox detection	Not directly observed.
String obfuscation, encryption, or packed content	Strong evidence was observed. ANY.RUN reported Base64 usage in both the script and PowerShell stages, character-substitution obfuscation via .replace(), AES cipher usage, IV and symmetric-key handling and dynamic assembly loading. The ProcMon log also captured a heavily obfuscated PowerShell command line using <code>iex([Text.Encoding]::Unicode.GetString([Convert]::FromBase64String(...)))</code> .

Suspicious APIs or code patterns related to evasion

Invoke-Expression, Base64 decoding, Unicode string reconstruction, AES-related operations, dynamic assembly loading, and execution-policy bypass behavior were all observed. These are consistent with hiding the real payload and making quick inspection harder.

Before-and-After Comparison

Artifact type	Change observed after execution
---------------	---------------------------------

Artifact type	Change observed after execution
Registry changes	The clearest registry change was PowerShell execution policy being written as Bypass, along with the Windows Script telemetry write. No confirmed persistence key such as a malicious Run value was proven by the before/after Autoruns comparison.
New or dropped files	Artifacts observed after execution included C:\Users\Public\Mands.png, C:\Users\Public\Vile.png, multiple __PSScriptPolicyTest_*.ps1 / *.psm1 files, and the modified ModuleAnalysisCache.
New services or scheduled tasks	No confirmed new malicious service or scheduled task was found.
Modified startup entries	No confirmed malicious startup entry was found in the manual Autoruns before/after comparison. Some differences appeared to be naming or display format changes for normal Windows / Group Policy components rather than true malware persistence.
Additional helper processes	The main helper processes were powershell.exe and conhost.exe, both launched from wscript.exe. The sandbox also showed supporting svchost.exe network activity, but the strongest malicious network actions were tied to powershell.exe.

Indicators of Compromise (IOCs)

Indicator type	Value
File hash	015545FF4D4C5D1FE37E7E1E6695CD4552C0B66A0241C399BC97A2CBB1EF4C5A
Registry key	HKCU\SOFTWARE\Microsoft\PowerShell\1\ShellIds\Microsoft.PowerShell Value observed: ExecutionPolicy = Bypass Also observed: HKCU\SOFTWARE\Microsoft\Windows Script\Settings\Telemetry\wscript.exe

Indicator type	Value
Dropped file	C:\Users\Public\Mands.png C:\Users\Public\Vile.png %LOCALAPPDATA%\Temp__PSScriptPolicyTest_*.ps1 %LOCALAPPDATA%\Temp__PSScriptPolicyTest_*.psm1
Service, task, or startup entry	No confirmed malicious service, task, or startup entry was isolated from the provided manual evidence.
Domain or IP address	ftp.ienco-gr.com 68.65.121.198 ip-api.com 208.95.112.1
Other host-based artifact	Obfuscated WScript.exe -> PowerShell.exe -> conhost.exe execution chain PowerShell ModuleAnalysisCache modification PowerShell command line using Base64 + IEX + Unicode decoding

Behavioral Summary and Reflection

Briefly summarize the malware's behavior, persistence, and evasion techniques

The sample behaved to usual characteristics of an Agent Tesla-related JavaScript launcher that used WScript.exe to execute an obfuscated PowerShell payload. The payload used Base64 encoding, character substitution, AES-related routines and dynamic assembly loading, which strongly suggests anti-analysis through obfuscation. It created temporary PowerShell artifacts, changed execution policy behavior, checked the external environment and made outbound FTP-related connections consistent with exfiltration.

What artifacts would be most useful for defenders trying to detect or remove this malware?

The most useful defender artifacts are the sample hash, PowerShell execution-policy registry modification, dropped files in C:\Users\Public, __PSScriptPolicyTest_* artifacts, modified ModuleAnalysisCache, WScript -> PowerShell -> conhost process chain and the malicious network indicators ftp.ienco-gr.com, 68.65.121.198, ip-api.com and 208.95.112.1. The obfuscated PowerShell command line pattern is also a strong indicator.

How did the malware's persistence or evasion increase its impact?

Even without confirmed persistence in this observed run, the evasion still increased impact by hiding the real payload behind an obfuscated script and PowerShell launcher. That makes the malware harder to understand quickly, harder to signature using only simple filenames and easier to blend into legitimate scripting activity. The use of FTP exfiltration and external-IP / hosting checks also suggests awareness of the environment and a goal of quietly stealing data once the payload is active.

Did ANY.RUN or another sandbox reveal behavior you did not initially observe manually?

Yes. ANY.RUN made several things much clearer than the VM capture alone, it identified the sample as Agent Tesla, flagged credential theft behavior, showed external IP / hosting checks through ip-api.com and showed FTP-based exfiltration to ftp.ienco-gr.com. It also surfaced dropped files and the PowerShell execution-policy modification more clearly than the initial VM run did.

After completing the form, choose **Print / Save as PDF** and select **Save as PDF** in your browser's print dialog.